

Relazione 2° Homework

Sistemi distribuiti

Gruppo:

- *Cristiano Pistorio 1000067656*
- *Giuseppe Romano 1000056390*

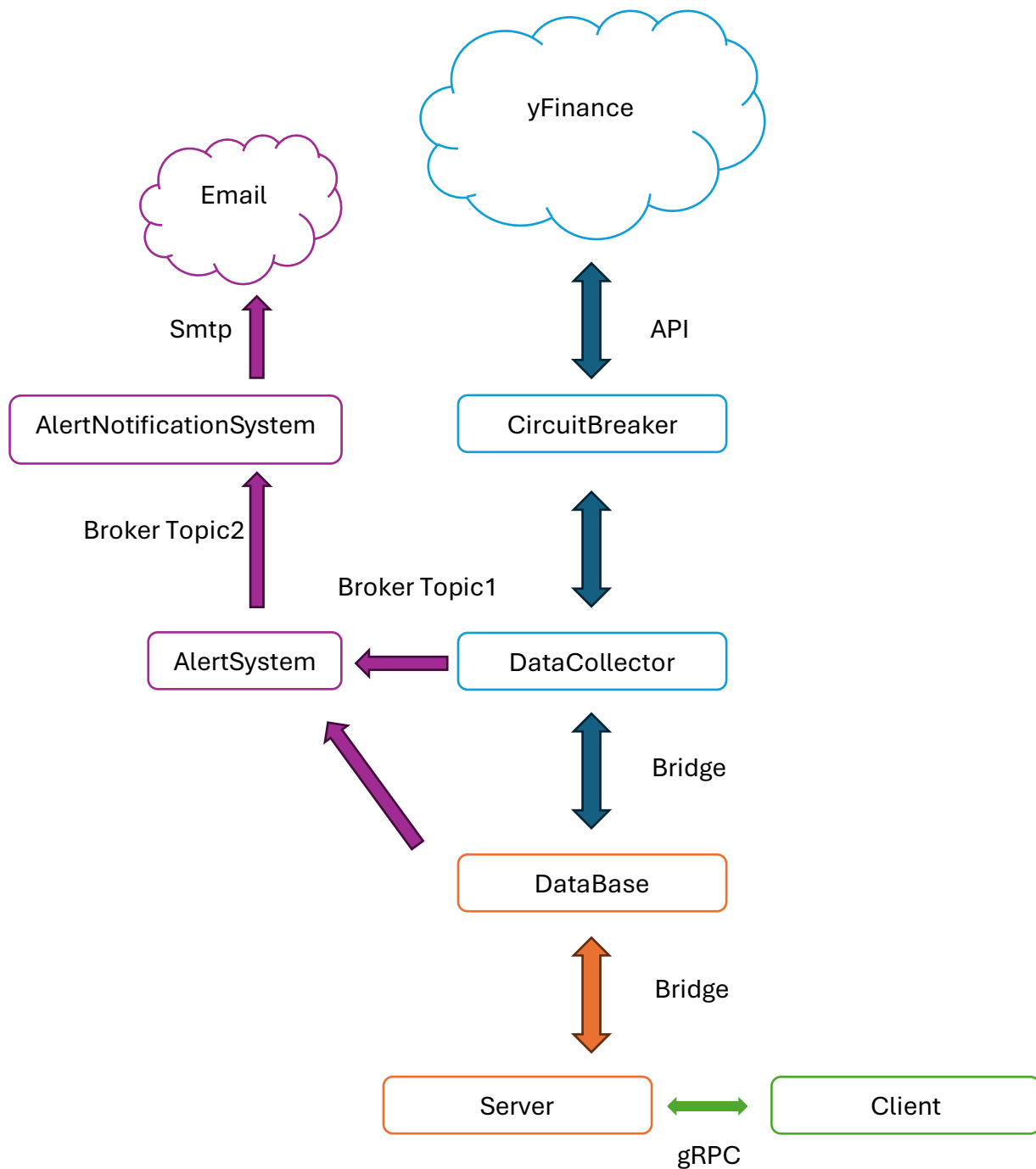
Sommario

Abstract	2
Diagramma architetturale.....	3
Server.....	4
Kafka.....	5
Container	6
Client gRPC	6
Pattern CQRS.....	6
Alert System	7

Abstract

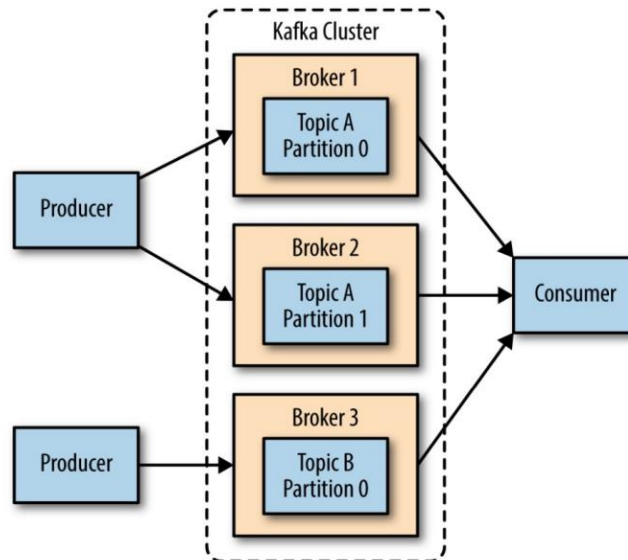
Nel server sono state integrate funzioni per gestire l'aggiunta, la modifica e l'eliminazione dei valori soglia per le notifiche. La registrazione utente è stata modificata per includere i parametri `low_value` e `high_value`, che definiscono il range di monitoraggio delle azioni. L'aggiornamento del ticker ora aggiorna anche le soglie, mentre l'aggiornamento delle soglie permette di modificare solo le soglie, utilizzando valori negativi per mantenere i valori precedenti. Apache Kafka è utilizzato per la gestione dei dati in tempo reale, con topic per l'organizzazione dei dati, broker per la memorizzazione delle partizioni, produttori per l'invio di messaggi e consumatori per la lettura dei messaggi. Nel progetto, il `datacollector` è un produttore che invia messaggi quando il database viene modificato, l'`alert system` è sia un produttore che un consumatore, e l'`alert system notification` è un consumatore che invia email di notifica. Sono state apportate modifiche al `docker_compose.yml` per includere Kafka e Zookeeper, con la creazione automatica dei topic abilitata. Il client gRPC è stato realizzato per testare l'applicazione, con un'interfaccia basica disponibile dal prompt dei comandi. Il pattern CQRS separa la gestione dei comandi dalle query, implementato nel file proto con `UserCommandService` per le operazioni di scrittura e `UserQueryService` per le operazioni di lettura.

Diagramma architetturale



Le modifiche attuate rispetto all'homework 1 sono evidenziate in viola.

Per rendere più leggibile il diagramma complessivo abbiamo semplificato il broker in una freccia; di seguito è presente un'immagine che spiega il funzionamento del broker kafka:



Server

Nel server sono state integrate delle funzioni per gestire l'aggiunta, la modifica e l'eliminazione dei valori soglia per cui poi inviare una notifica.

Le operazioni implementate o modificate sono:

- Registrazione utente è stata modificata
Rimane simile alla funzione non modificata, la differenza sta solo nel fatto che si necessita di due parametri nuovi in input: `low_value`, `high_value`.
Questi parametri servono per tenere traccia del range che l'utente vuole monitorare; se il valore dell'azione scende sotto `low_value` o sale sopra `high_value` verrà inviata una notifica all'utente. Ovviamente il `low_value` non potrà essere maggiore di `high_value`.
L'utente potrebbe anche voler non tracciare l'azione oppure porre solamente uno dei due limiti, per fare ciò abbiamo caratterizzato il valore di soglia 0.0. Se nel db è presente 0.0 in una delle due soglie o in entrambe vorrà dire che non si dovrà tracciare quella o quelle soglie (ad esempio un utente potrebbe voler essere notificato SOLAMENTE se la soglia supera 100, in questo caso `low_value` sarà 0 e `high_value` sarà 100, vale anche il duale).
- Aggiornamento ticker è stata modificata
Non ha senso tenere le soglie precedenti quando si aggiorna il ticker; quindi, abbiamo deciso di aggiornare anche le soglie quando viene invocata questa funzione.
- Aggiornamento soglia è stata creata

L'utente potrebbe voler aggiornare solo le soglie, può fare questo attraverso questa funzione.

L'unica cosa da attenzionare in questo caso è che abbiamo gestito l'intenzione dell'utente di voler modificare una o entrambe le soglie impostate, per farlo l'utente dovrà inserire un valore negativo che assume il significato di "tenere il valore precedente", ovviamente se uno dei due valori rimane uguale e l'altro cambia si dovrà prima di completare l'operazione verificare se il vincolo $low < high$ è ancora rispettato.

Kafka

Apache Kafka è una piattaforma distribuita di messaggistica e streaming, progettata per gestire grandi quantità di dati in tempo reale.

Gli elementi fondamentali di Kafka e che abbiamo adoperato nel nostro progetto sono i seguenti:

- **Topic:** Canali logici in cui i dati vengono organizzati e distribuiti. Ogni topic è suddiviso in **partizioni**, che garantiscono scalabilità e ordinamento locale dei messaggi. Ogni messaggio ha un **offset** che ne identifica la posizione nella partizione.
- **Broker:** Nodo del cluster Kafka responsabile della memorizzazione delle partizioni di uno o più topic. I broker collaborano per distribuire i dati e garantire la replica.
- **Produttore:** Componente che invia messaggi ai topic. Può specificare una partizione o affidarsi al bilanciamento automatico.
- **Consumatore:** Componente che legge i messaggi da uno o più topic. I consumatori possono appartenere a **gruppi di consumo**, permettendo la divisione del lavoro tra più istanze.

Kafka garantisce una comunicazione distribuita e scalabile tra produttori e consumatori, con i topic che fungono da punto di incontro per la gestione dei dati.

Come visibile anche dal diagramma architetturale nel nostro contesto il datacollector diventa un produttore che invia un messaggio solo quando il db viene modificato.

L>alert system sarà sia un produttore che un consumatore. Questo è un consumatore perché riceve il messaggio dal datacollector, una volta ricevuto questo scandisce il db alla ricerca di violazioni del range, se le riscontra invia un messaggio all>alert system notification, per questo l>alert system è anche produttore., inoltre aggiornerà le informazioni di quel ticker per l'utente modificando la soglia al valore che ha causato la violazione.

Ultimo entità a fare uso di kafka è ovviamente l'alert system notification che è consumatore del messaggio prodotto dall'alert system, che ha il compito di inviare l'email all'utente per notificare la violazione di una soglia.

Container

Sono state attuate delle modifiche al docker_compose.yml, abbiamo aggiunto le istruzioni per generare il microservizio kafka attraverso la sua immagine. Per far funzionare kafka è necessario anche utilizzare zookeeper e quindi generare questo servizio a partire sempre dalla sua immagine.

Per far dialogare il tutto è necessario aggiungere kafka e zookeeper alla rete.

Per far partire tutto nell'ordine corretto abbiamo aggiunto le adeguate dipendenze.

Dettaglio da attenzionare è che nel servizio di kafka abbiamo impostato la creazione automatica dei topic, per evitare errori e per non appesantire il sistema con ulteriori servizi. Abbiamo fatto ciò attraverso il comando:

```
KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
```

Client gRPC

Il client è stato realizzato per testare l'applicazione, il suo funzionamento prevede un'interfaccia basica disponibile dal prompt dei comandi.

Si ha un menù dal quale si possono eseguire tutte le operazioni specificate nella consegna ed alcune operazioni che abbiamo aggiunto per visualizzare velocemente lo stato del sistema.

```
Menu:
1. Registra Utente
2. Aggiorna ticker utente
3. Modifica valori di soglia
4. Elimina Utente
5. Mostra Tutti i Dati
6. Mostra Ultimo Valore Azione
7. Calcola Valore Medio Azione
8. Elimina Dati per Tempo
9. Esci
Inserisci la tua scelta: █
```

Pattern CQRS

CQRS (Command and Query Responsibility Segregation) è un pattern architetturale che separa la gestione dei **comandi** (azioni che modificano lo stato del sistema) dalla gestione delle **query** (richieste di lettura del sistema).

Il pattern è stato implementato attraverso il file proto, qui sono state separate le responsabilità:

- **UserCommandService** gestisce le operazioni di scrittura, come la registrazione, l'aggiornamento e la cancellazione degli utenti.
- **UserQueryService** gestisce le operazioni di lettura, come ottenere tutti i dati o il valore delle azioni.

È stato modificato il file [user.proto](#).

Alert System

Nel capitolo dedicato a Kafka, il nostro sistema di allerta è descritto come sia un produttore che un consumatore. L>alert system attende un messaggio dal data collector, che segnala quando avviene una modifica al database. Tuttavia, il data collector non invia direttamente le modifiche al database; invece, l>alert system esamina il database per rilevare eventuali modifiche di suo interesse.

Dopo aver scandagliato il database, l>alert system verifica se ci sono valori di azioni che superano le soglie impostate dall'utente. Se ciò accade, invia un messaggio al sistema di notifica, che si occupa di inviare una mail di notifica tramite protocollo SMTP.

Per evitare l'invio di raffiche di email, la soglia superata dall'utente viene modificata. Ad esempio, se la soglia massima per un ticker X è 100 e il valore dell'azione raggiunge 101, rimanendo stabile a tale valore, la variabile high value dell'utente viene aggiornata a 101. In questo modo, ulteriori email vengono inviate solo se il valore dell'azione continua a crescere.